

Roken Redux

Gary Nan Tie

Jun 28, 2026

Abstract

Roken, a novel LLM algorithm characterized by scalable parallel transformer inference, structured on pre-trained ImplicitRNN state machines.

Synopsis

We revisit and refine **Roken**, recurrent Noken [1], a dynamic implicit deep learning model of attention, designed to integrate state tracking into large language model next-token and sequence-to-sequence learning. As we shall see, the LLM architecture of Roken is characterized by novel scalable parallel transformer inference structured on pre-trained ImplicitRNN state machines.

Recall that a state machine begins in an initial state h_0 , that gets updated with each successive input t_i , via $h_i = S(h_{i-1}, t_i)$ where S is a transition function, and then outputs $o_{i+1} = O(h_i)$ where O is an output function. (The right index shift is for autoregressive continuation.)

A word embedding is typically a representation of a word in a vector space. The representation encodes the meaning of the word in such a way that the words that are closer in the vector space are expected to be similar in meaning. For example, the word embedding word2vec [2], captures information about the meaning of a word based on the surrounding words.

The recurrent architecture of ImplicitRNN [3] enables state tracking. First, we pre-train a vector ImplicitRNN with fixed points v , to learn a word embedding. The resultant state machine captures word semantics. Next, a token ImplicitRNN with fixed points k , is used to model a second state machine with hidden states h , contextual token representations, and output O , next-token, a function of h . Thus the word embedding creates a fixed point pairing (k, v) , to be now utilized by parallel attention layers as follows:

- the similarity of h across k to date generates probabilities
- which in turn creates a probability weighted sum of v
- that is then updated by applying a ReLU feedforward network (FFN) pointwise

The updated h is passed as input h to the next attention layer and so on. The stacked layers compose the nonlinear updates. For the final layer we output the token whose k is most similar (probable under a distribution β) to the last updated h , as the next-token O . The objective of the next-token learning problem is to learn ImplicitRNN and FFN parameters so as to minimize output perplexity regularized by average Renyi α -entropy of β .

For sequence-to-sequence learning, we solve the next-token problem for both the target and source tokens thereby learning contextual token representations for target and source. We can then directly compare their state similarity to determine a best translation based on meaning rather than surface syntax alone.

In a sense, the first state machine learns semantics, the second syntax, with transformer inference then synthesizing the two by parallel attention layer composition. Moreover, low displacement rank operators and robust implicit networks via non-Euclidean contractions [4] ensure fast tractable chunk scaling via mini-batch SGD. So, we can have our cake, and eat it fast too!

Perhaps the deepest implication is epistemological. Perplexity optimization assumes language understanding emerges from predicting what comes next. The Roken formulation distinguishes two separate learning objectives:

- syntactic prediction (Perplexity)

- semantic certainty (Renyi α -entropy)

This implies that "understanding" is not merely next-token prediction but also the reduction of semantic uncertainty over latent meanings. Viewed this way, a Roken LLM is moving from a purely statistical theory of language toward a dynamical-semantic theory of language. The hidden state is no longer just a computational artifact. It becomes an evolving belief state whose entropy reflects how confidently the model has resolved meaning. In this sense, the ramification is not merely a better recurrent LLM, it is a framework in which syntax, semantics, memory, and reasoning become different aspects of a single dynamical system evolving on latent state manifolds. Meaning matters!

Rokan Redux - 6/28/26

State Machines

Input: target tokens $t_1, \dots, t_m \in \mathbb{R}^P$

Initial state: $h_0 = 0 \in \mathbb{R}^q$

Next state: $h_i = S(h_{i-1}, t_i) \in \mathbb{R}^q, i \in [m]$

Output: $o_{i+1} = O(h_i) \in \mathbb{R}^P, i \in [m]$

where S is a transition function and

O is an output function.

+

Let $\text{Emb} : \mathbb{R}^P \rightarrow \mathbb{R}^{P'}$
 $t_i \mapsto t'_i, i \in [m]$

be a word embedding, like word2vec.

We use ImplicitRNN [3] to construct two
state machines (see over).

Token

Implicit RNN

$$h_0 = 0 = k_0 \in \mathbb{R}^q$$

$$u_i = \begin{pmatrix} t_i \\ h_{i-1} \end{pmatrix} \in \mathbb{R}^n$$

$$i \in [m], \quad r = p + q$$

$$(+) \quad \begin{cases} k_i = \varphi(Ak_i + Bu_i) \\ h_i = Ck_i + Du_i \end{cases}$$

$$=: \mathcal{S}(h_{i-1}, t_i)$$

Vector

Implicit RNN

$$h'_0 = 0 = v_0 \in \mathbb{R}^{q'}$$

$$u'_i = \begin{pmatrix} t'_i \\ h'_{i-1} \end{pmatrix} \in \mathbb{R}^{r'}$$

$$i \in [m], \quad r' = p' + q'$$

$$(\#) \begin{cases} v_i' = \varphi'(A'v_i + B'u_i') \\ h_i' = C'v_i + D'u_i' \end{cases}$$

where $A = J - \text{diag}(|J| \mathbb{1}_g) + \gamma \mathbb{I}_g$, $0 < \gamma < 1$,

makes A well-posed for φ (similarly for A') [4].

To complete the state machine constructions

we go on to define their respective output

functions.

Let $\alpha'_{ij} = \text{softmax score}(h'_i, v_j)$
 $j \in [l]$

where score is a similarity function, like a reproducing

kernel, and let $j_i^{\alpha'} = \arg \max_{j \in [l]} \alpha'_{ij}$

Define $o'_{i+1} = \mathcal{O}'(h'_i) := t_{j_i^{\alpha'}}$, $i \in [m]$.

Note that we have:

tokens $t_1, \dots, t_i, o'_{i+1}$

vectors $t'_1, \dots, t'_i, t'_{i+1}$

Let Cos denote cosine distance

$$\text{ie } \text{Cos}(a, b) = \frac{\langle a, b \rangle}{\|a\| \|b\|}$$

In (†), learn matrices A', B', C', D' and fixed points

v_i to minimize $\sum_{i=1}^{m-1} \text{Cos}(\text{Emb}(o'_{i+1}), t'_{i+1})$

ie vector Implicit RNN learns $\text{Emb} : \mathbb{R}^P \rightarrow \mathbb{R}^{P'}$
 $t_i \mapsto t'_i \quad i \in [m]$

+

Attention layer(s)

For $i \in [m]$, let $\alpha_{ij} \triangleq \text{softmax score}(h_i, k_j)$

$$h_i^0 \triangleq \sum_{j=1}^i \alpha_{ij} v_j$$
$$h_i^1 \triangleq \text{ReLU}(W^1 h_i^0 + b^1)$$

weight matrix W^1
bias vector b^1

Repeat L times

First layer: Start with h_i from Token ImplicitRNN(t)
Update h_i to h_i^1

Second layer: Let h_i be h_i^1 from the previous layer

That is define $h_i := h_i^1$ from layer 1.

Update h_i as before.

•
•
•

L th layer

Stack L layers and let $\Theta = \{(W^l, b^l)\}_{l=1}^L$

denote the ReLU FeedForward network parameters

and let h_i^L denote the final L -th update of h_i .

—

Let $\beta_{ij} = \underset{j \in [i]}{\text{softmax score}}(h_i^L, k_j)$

and $j_i^\beta = \underset{j \in [i]}{\arg \max} \beta_{ij}$

Define $o_{i+1} = \mathbb{O}(h_i) := t_{j_i^\beta}$

This completes the construction of state machines

$(\mathcal{S}, \mathbb{O})$ and $(\mathcal{S}', \mathbb{O}')$

—

Define the average Renyi α -entropy of $\{t_i\}_{i=1}^m$

by $E(\{t_i\}_{i=1}^m) = \frac{1}{m} \sum_{i=1}^m H_\alpha^i$

where $H_\alpha^i \triangleq H_\alpha(\{\beta_{ij}\}_{j=1}^i)$

and H_α is Renyi α -entropy.

—

Next-token learning problem*

Let $\lambda > 0$, learn matrices A, B, C, D

and fixed points k_i in $(\dagger)$ and ReLU

feedforward network parameters Θ in attention

layers to minimize:

$$\left\{ \text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\}_{i=1}^m) \right\}$$

+

Next-token prediction

Initialization: let t_1 = the empty token or a seed token

Given $[t_1]$, define $t_2 = o_2$

Given $[t_1, t_2]$, define $t_3 = o_3$, and so on,

autoregressively generating next-token continuation

+

Sequence-to-Sequence learning*

Input: target tokens $t_1, \dots, t_m$

source tokens $t'_1, \dots, t'_{m'}$

For $\{t_i\}_{i=1}^m$ and $\{t'_j\}_{j=1}^{m'}$ solve their respective

next-token learning problem to ^{find} contextual representations

$\{h_i^L\}_{i=1}^m$ and $\{h_j^{L'}\}_{j=1}^{m'}$.

Define $\delta_{ij} = \underset{j \in [m']}{\text{softmax score}}(h_i^L, h_j^{L'})$, $i \in [m]$

and let $j_i^\delta = \underset{j \in [m']}{\arg \max} \delta_{ij}$.

Given target t_i , $i \in [m]$

define source output $o_i = t'_{j_i^\delta}$

So $t_1, \dots, t_m$ is translated to $o_1, \dots, o_m$

where $o_i \in \{t'_j\}_{j=1}^{m'}$.

—

* In the learning problems, the number of input tokens is the mini-batch size for stochastic gradient descent. On each corpus chunk, we use mini-batch SGD to jointly learn ImplicitRNN and attention layer Feedforward network parameters, so as to minimize perplexity, regularized by average Renyi α -entropy.

References

[1] 'Implicitly learning token semantics'

Gary Nan Tie, 3 Mar 2026

DOI: 10.13140/RG.2.2.30432.34566/1

[2] 'Efficient Estimation of Word Representations in Vector Space'

Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean

<https://arxiv.org/abs/1301.3781>

[3] 'The Extrapolation Power of Implicit Models'

Juliette Decugis, Alicia Y. Tsai,

Max Emerling, Ashwin Ganesh, Laurent El Ghaoui

arXiv:2407.14430v1 [cs.LG] 19 Jul 2024

[4] 'Robust Implicit Networks via Non-Euclidean Contractions'

Saber Jafarpour, Alexander Davydov,

Anton V. Proskurnikov, Francesco Bullo

arXiv:2106.03194v6 [cs.LG] 26 Jan 2022